Лабораторная работа № 4

Распределенные базы данных Oracle Database 11g Express Edition

1. Цель работы

Изучение средств Oracle для создания распределенных баз данных и хранилищ данных.

2. Задачи

Изучение принципов функционирования распределенных баз данных, использования представлений и материализованных представлений в распределенных базах данных и хранилищах данных, создание SQL-запросов на выборку данных с объединением таблиц в распределенной базе данных.

1. Теоретическая часть

3.1. Распределенные базы данных

Распределённая база данных (РБД) — база данных, составные части которой размещаются в различных узлах компьютерной сети в соответствии с каким-либо критерием. Распределенные данные представляют собой РБД, только если они связаны в соответствии с некоторым структурным формализмом (таким как реляционная модель), а для доступа к ним имеется единый высокоуровневый интерфейс.

Распределенная база данных (система распределенных баз данных) состоит из набора узлов, связанных коммуникационной сетью, в которой:

- каждый узел — это полноценная СУБД сама по себе, но
- узлы взаимодействуют между собой таким образом, что пользователь любого из них может получить доступ к любым данным в сети так, как будто они находятся на его собственном узле.

Чаще всего предполагается, что узлы физически распределены (а возможно, и географически,), хотя в действительности достаточно того, чтобы они были распределены логически.

Необходимые функциональные возможности для организации РБД предоставляет логическое расширение локальной СУБД – **распределенная система управления базами данных** (РСУБД).

Фундаментальный принцип создания распределенных баз данных: **для пользователя распределенная система должна выглядеть так же, как нераспределенная система.**

Другими словами, пользователи распределенной системы должны иметь возможность действовать так, как если бы система не была распределена. Все проблемы распределенных систем относятся или должны относиться к внутренним проблемам или проблемам реализации, а не к внешним проблемам или проблемам пользовательского уровня.

Сформулированный фундаментальный принцип имеет следствием определенные дополнительные правила или цели. Таких целей всего двенадцать:

1. **Локальная независимость.** Узлы в распределенной системе должны быть независимы, или автономны. Локальная независимость означает, что все операции, управление данными и доступ к ним остаются в ведении каждого узла.
2. **Отсутствие опоры на центральный узел.** Локальная независимость предполагает, что все узлы в распределенной системе должны рассматриваться как равные. Поэтому, в частности, не должно быть никаких обращений к "центральному" или "главному" узлу с целью получения некоторого централизованного сервиса. Таким образом, вторая цель на самом деле является следствием первой цели — если первая цель достигнута, то вторая цель также заведомо достигнута.
3. **Непрерывное функционирование.** В общем случае преимущество распределенных систем состоит в том, что они должны предоставлять более высокую степень надежности и доступности. Надежность распределенных систем повышается за счет того, что они могут непрерывно функционировать (в сокращенном варианте) даже в случаях отказов части их компонентов, таких как отдельный узел. Доступность распределенных систем также повышается — частично по тем же причинам, а также благодаря возможности дублирования данных.
4. **Независимость от расположения** (прозрачность расположения): пользователи не должны знать, где именно данные хранятся физически и должны поступать так (по крайней мере, с логической точки зрения), как если бы все данные хранились на их собственном локальном узле. Благодаря независимости от расположения упрощаются пользовательские программы и терминальные операции.
5. **Независимость от фрагментации.** Система поддерживает независимость от фрагментации, если данная таблица может быть разделена на части или *фрагменты* при организации ее физического хранения. Фрагментация желательна для повышения производительности системы. В этом случае данные могут храниться в том месте, где они чаще всего используются, что позволяет достичь локализации большинства операций и уменьшения сетевого трафика. Существует два основных вида фрагментации: *горизонтальная* и *вертикальная*; они соответствуют реляционным операциям выборки (отбору строк таблицы по условию) и проекции (выбору столбцов таблицы) соответственно. В более общем случае фрагмент можно представить в виде произвольного сочетания операций выборки и проекции. Восстановление исходной таблицы из ее фрагментов выполняется с помощью соответствующих операций соединения (для вертикальных фрагментов) и объединения (для горизонтальных). Простота выполнения фрагментации и восстановления — это две основные причины, по которым распределенные базы данных должны быть именно реляционными.
6. **Независимость от репликации.** Система поддерживает репликацию данных, если данная таблица или ее фрагмент могут быть представлены в виде многократно хранимой копии, причем каждая копия может храниться на своем узле. Преимущества репликации состоят в том, что она может повысить производительность (доступ к данным можно осуществлять локально) и надежность (данные доступны даже в случае отказа одного из узлов, на котором хранится копия). Следовательно, такие вопросы, как безопасность, целостность и представление локальных данных в памяти, остаются под контролем и в пределах компетенции локального узла.

отдельными копиями или *репликами*, которые хранятся на нескольких отдельных узлах. Репликация желательна по крайней мере по двум причинам:

- репликация способна обеспечить более высокую производительность, поскольку приложения смогут обрабатывать локальные копии вместо того, чтобы устанавливать связь с удаленными узлами.
- наличие репликации может также обеспечивать более высокую степень доступности, поскольку любой реплицируемый объект остается доступным для обработки (по крайней мере для выборки данных), пока хотя бы одна реплика в системе остается доступной.

Главным недостатком репликации является то, что если реплицируемый объект обновляется, то и **все его копии** должны быть обновлены. Для пользователей должна быть создана такая среда, чтобы они, по крайней мере с логической точки зрения, могли считать, что в действительности данные не дублируются.

7. **Обработка распределенных запросов.** Суть в том, что для выполнения запроса может потребоваться обращение к нескольким узлам. В такой системе может быть много возможных способов пересылки данных, позволяющих выполнить рассматриваемый запрос. Поэтому крайне важно, чтобы была найдена эффективная стратегия. Например, запрос для объединения таблицы Rx, хранимой на узле X, и таблицы Ry, хранимой на узле Y, может быть выполнен посредством пересылки таблицы Rx на узел Y или таблицы Ry на узел X, или обеих таблиц на какой-либо узел Z и т.п. Время ответа в каждом случае различно и изменяется в широких пределах. Нередко подобные запросы выполняются в течение нескольких часов.
8. **Управление распределенными транзакциями.** В распределенной системе отдельная транзакция может включать в себя выполнение кода на многих узлах, в частности это могут быть операции обновления, выполняемые на нескольких узлах. Каждая транзакция содержит несколько агентов, где под агентом подразумевается процесс, который выполняется для данной транзакции на отдельном узле. Система должна знать, что два агента являются элементами одной и той же транзакции. Например, два агента, которые являются частями одной и той же транзакции, не должны оказываться в состоянии взаимной блокировки. Такого результата можно достичь с помощью протокола *двухфазной фиксации транзакции*.
9. **Аппаратная независимость.** Желательно иметь возможность запускать одну и ту же СУБД на различных аппаратных платформах и добиться, чтобы различные машины участвовали в работе распределенной системы как равноправные партнеры.
10. **Независимость от операционной системы.** Достижение этой цели частично зависит от достижения предыдущей и не требует дополнительного обсуждения.
11. **Независимость от сети.** Если система имеет возможность поддерживать много принципиально различных узлов, отличающихся оборудованием и операционными системами, очевидно, необходимо, чтобы она также поддерживала ряд типов различных коммуникационных сетей.
12. **Независимость от типа СУБД.** Необходимость строгой однородности РБД относительно СУБД различных узлов не должна быть обязательной.

Распределенные системы вполне могут быть, по крайней мере в некоторой степени, *неоднородными*. Поддержка неоднородности весьма заманчива. На практике современное программное обеспечение обычно используется не только на многих различных компьютерах и в среде многих различных операционных систем. Оно довольно часто используется и с различными СУБД, и было бы очень хорошо, если бы различные СУБД можно было включить в распределенную систему. Это возможно, если все узлы РБД поддерживают официальный стандарт языка SQL и стандартные протоколы обмена по сети передачи данных.

3.2. Хранилища данных

Хранилище данных (data warehouse) — это интегрированное представление основного массива данных организации. Хранилище данных имеет четыре основных признака, отличающих его от «обычной» базы данных (оперативной системы):

1. **Ориентация на конкретный предмет.** Хранилище данных организуется для поддержки специфических бизнес-процессов либо определенных областей знаний.
2. **Интеграция данных.** Хранилище данных должно быть интегрированным, чтобы обеспечить взаимосвязь разных направлений деятельности, реализуемых в организации. Хранилище данных должно представлять собой единое место в организации, где сотрудники могут анализировать информацию с разных точек зрения, поэтому интеграция данных абсолютно необходима. Например, возможно произвести сравнение объемов продаж с результатами изобретательской деятельности.
3. **Постоянство данных.** Хранилище данных представляет собой статическую систему. Оно не является таким изменчивым, как оперативная система. Информация регулярно добавляется в хранилище данных, но после загрузки она не модифицируется. Таким образом, информация загружается в хранилище данных только один раз, но может многократно считываться.
4. **Временная привязка данных.** Мощь хранилища данных основана на том, что загружаемая в хранилище информация соответствует конкретным моментам или интервалам времени. Компонент данных может быть привязан к дате продаж, а также к ежедневным, еженедельным, месячным и годовым итогам продаж. Хранилище данных предоставляет пользователям возможность в любой момент времени выполнять анализ информации, используя заранее обобщенные данные по разным временным интервалам.

Размерностное проектирование хранилищ данных. В настоящее время наиболее популярной методикой проектирования считается *размерностное проектирование хранилищ данных* (dimensional data warehousing). Размерностное хранилище данных основано на двух базовых сущностях: на фактах и размерностях.

Таблицы фактов (fact tables) содержат информацию, которую мы хотим измерять. Например, мы можем иметь таблицу фактов, содержащую сведения о ежедневных продажах в компании.

Таблицы размерностей (dimension tables) могут содержаться сведения о клиентах, продавцах, товарах, складах, а также географические данные.

Вместе таблицы фактов и размерностей образуют структуру, называемую *звездообразной схемой* (star schema). Это

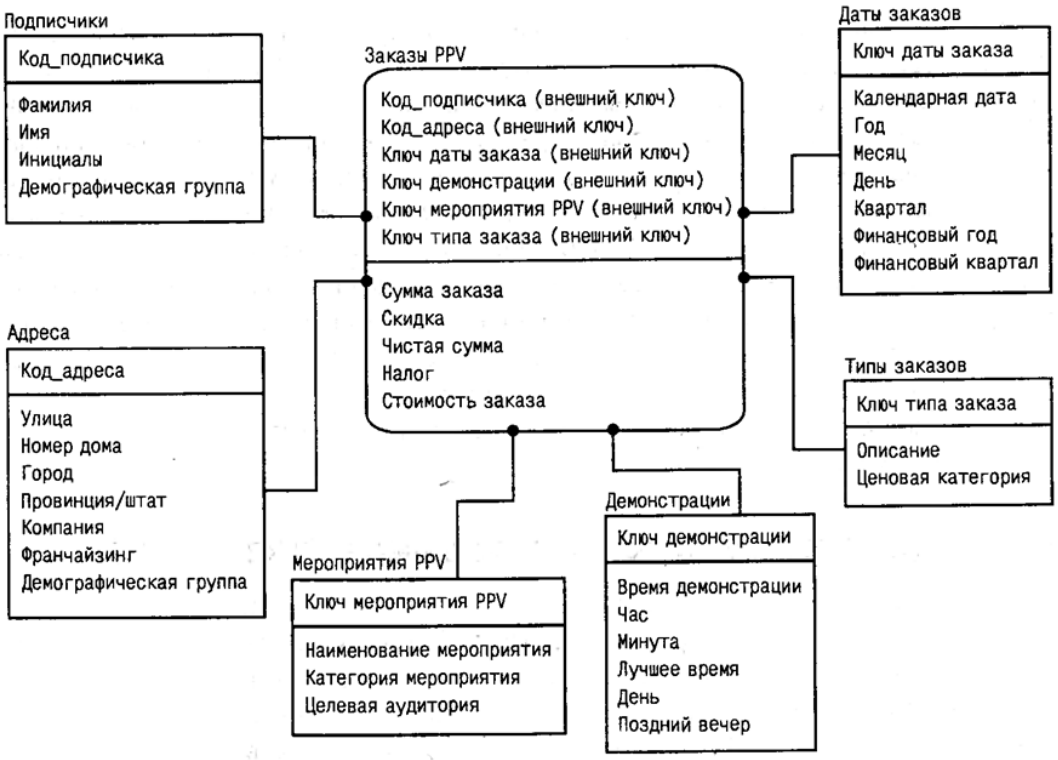


Рис. 4.1. Звездообразная схема хранилища данных

Здесь таблица фактов – Заказы PPV, таблицы размерностей – все остальные. Эта схема хорошо отражает все четыре признака хранилища данных.

Разделение данных в хранилищах. Данные, которые находятся в хранилище, очень хорошо подходят для разделения. Помимо двух основных видов фрагментации – горизонтальной и вертикальной, могут использоваться и другие. Некоторые часто используемые виды разделения данных в распределенных хранилищах показаны на рис. 4.2:

- представленная звездообразная структура хранит данные в схемах двух пользователей – HR (таблица HR.EMPLOYEES) и OE (таблицы OE.CUSTOMERS, OE.PRODUCT_INFORMATION, OE.ORDERS, OE.ORDER_ITEMS);
- таблицы размерностей (HR.EMPLOYEES, OE.CUSTOMERS, OE.PRODUCT_INFORMATION) могут располагаться на разных узлах распределенного хранилища;
- таблица фактов разделена на две таблицы – родительскую OE.ORDERS и дочернюю OE.ORDER_ITEMS, которые также могут располагаться на другом узле, отдельно от таблиц размерностей.

Для объединения таблиц OE.ORDERS и OE.ORDER_ITEMS в единую таблицу фактов можно использовать представление или материализованное представление.

Описание связей между таблицами на рис. 4.2, включая имя родительской таблицы и имена связанных столбцов таблиц, можно получить из результата выполнения п.3 задания к лабораторной работе № 2.

Это схема создер

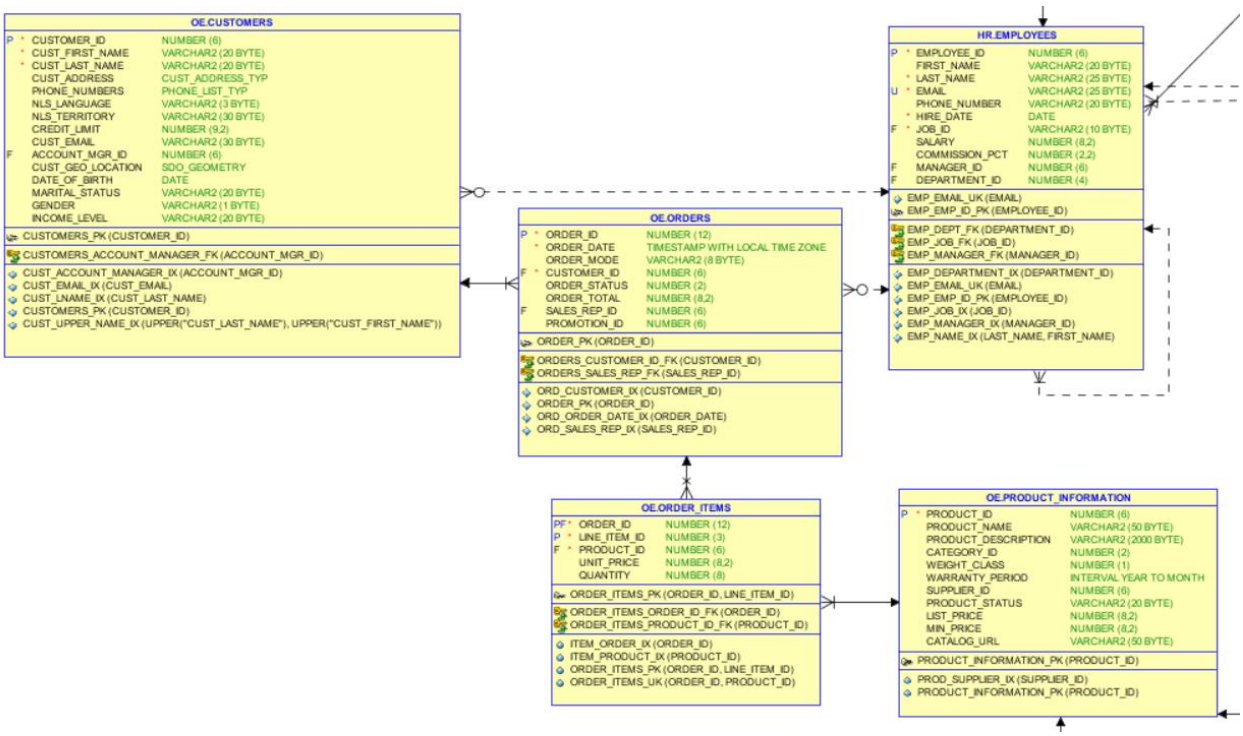


Рис. 4.2. Звездообразная схема хранилища данных с разделенной таблицей фактов

3.3. Средства Oracle для работы с распределенными базами данных и хранилищами данных

3.3.1. Связь базы данных Oracle

Связь базы данных Oracle (database link) — это объект схемы пользователя базы данных Oracle, представляющий собой одностороннее соединение локальной базы данных с удаленной базой данных. Связь всегда односторонняя. Пользователи удаленной базы не могут применять ее для подключения к локальной базе — вместо этого они должны создать отдельную связь базы данных.

Объект Database Link (связь) базы данных позволяет получать доступ к разным базам данных через учетную запись пользователя удаленной базы; вы сами не обязаны быть зарегистрированным пользователем удаленной базы данных. Привилегии в этой базе данных будут идентичны привилегиям пользовательской учетной записи, которая применялась для создания связи. Связи баз данных удобны, когда необходимо запросить таблицу в распределенной базе данных, или даже вставить данные из таблицы другой базы в собственную локальную таблицу. Связи баз данных позволяют пользователям обращаться к множеству баз данных как к единой логической базе данных.

Связи баз данных могут быть *приватные* и *общедоступные*.

Приватная связь (Database Link) базы данных принадлежит пользователю, который ее создал. Только пользователь, создавший приватную связь базы данных, может использовать ее. Когда другой пользователь попытается воспользоваться этой связью удаленной базы данных, произойдет ошибка – будет выдано сообщение «*ORA-02019: connection description for remote database not found*» (не найдено описание подключения для удаленной базы данных).

Для создания приватной связи базы данных пользователь должен иметь привилегию *CREATE PRIVATE DATABASE LINK* в локальной базе данных.

Общедоступная связь (Public Database Link) базы данных позволяет любому пользователю обращаться к объектам удаленной базы данных. Как только общедоступная связь создана, любой пользователь может с ее помощью входить в удаленную базу. Пользователь может обращаться к удаленной базе, даже не будучи ее зарегистрированным пользователем, потому что применяет общедоступную связь, что избавляет его от необходимости указывать комбинацию имени и пароля пользователя для доступа к объектам удаленной базы.
Для создания общедоступной связи базы данных пользователь должен иметь привилегию *CREATE PUBLIC DATABASE LINK* в локальной базе данных.
Подключение к удаленной базе данных. Простейший и наиболее легкий способ установки подключения Oracle к удаленной базе данных использует так называемые *локальные именования*. В этом методе используются **имена служб** и их **дескрипторы соединения**, которые сохраняются в локальном файле конфигурации **tnsnames.ora**. По умолчанию этот файл всегда находится в каталоге $ORACLE_HOME/network/admin **удаленного компьютера**. Здесь $ORACLE_HOME – каталог установки программного обеспечения сервера Oracle (например, C:/oraclexe/app/oracle/product/11.2.0/server).
В сети каждая база данных идентифицируется по ее имени службы, для указания которого служит параметр инициализации SERVICE_NAME. Термин «дескриптор соединения» в Oracle используют для обозначения объединенной спецификации двух обязательных компонентов подключения к базе данных: имени службы базы данных и ее адреса. Часть адреса дескриптора соединения содержит три компонента: коммуникационный протокол, используемый для соединения, имя хоста и номер порта. Стандартным протоколом является TCP/IP или TCP/IP с SSL (Secure Sockets Layer — уровень защищенных сокетов). На компьютерах Windows по умолчанию используется порт 1521. Пример типичного дескриптора соединения:

```
(DESCRIPTION
        (ADDRESS=(PROTOCOL=tcp) (HOST=ALPHA6) (PORT=1521))
                (CONNECT_DATA=
                (SERVICE_NAME=XE)))
```

С дескриптором соединения тесно связан *идентификатор соединения*. Ниже показан пример, демонстрирующий отображение идентификатора соединения XE:

```
XE =
  (DESCRIPTION =
    (ADDRESS = (PROTOCOL = TCP)(HOST = DESKTOP-B0DUQAC)(PORT = 1521))
    (CONNECT_DATA =
      (SERVER = DEDICATED)
      (SERVICE_NAME = XE)
    )
  )
```

В этом примере инструкция (SERVER = DEDICATED) указывает, что запросы клиентов будут обслуживаться выделенным сервером.
Подключение к базе данных выполняется путем указания **строки соединения**. Строка соединения содержит сочетание имени пользователя и пароля и идентификатор соединения. Один из наиболее часто используемых идентификаторов соединения — имя сетевой службы, которое представляет собой простое имя службы базы данных. Синтаксис строки соединения выглядит следующим образом:

имя_пользователя/пароль@[//]хост[:порт][/имя_службы]

В этой синтаксической конструкции:

- // - этот элемент не обязателен;
- хост - этот параметр обязателен. Можно указать либо символическое имя хоста, либо IP-адрес сервера, на котором расположена удаленная база данных;
- порт - этот параметр не обязателен. Если порт не указан, по умолчанию используется порт 1521;
- имя_службы - этот параметр указывает имя службы базы данных (по умолчанию оно совпадает с именем хоста) и является необязательным. Если имя хоста и имя сервера базы данных идентичны, этот параметр можно опустить. Если же они не совпадают, понадобится указать допустимое имя службы для идентификации базы данных.

Следующий пример демонстрирует подключение к базе данных XE, расположенной на сервере ALPHA6. Подключение осуществляется в окне утилиты SQL*Plus (SQL Command Line):

SQL > connect oe/oe@//ALPHA6:1521/XE;

Создание общедоступной связи Public Database Link с помощью мастера Oracle SQL Developer:

1. В навигаторе *Connections* раскрыть соединение пользователя *sys*. Раскрыть узел *Public Database Links*. В списке связей **удалить все связи с именами, отличными от DBMS_CLRDBLINK**: выбрать удаляемую связь -> контекстное меню -> Drop… -> Принять -> OK.
2. Запустить мастер создания общедоступной связи: узел *Public Database Links* -> контекстное меню -> New Public Database Link… -> удостовериться в наличии флажка *Public* -> в поле *Name* задать уникальное имя создаваемой связи (например, по имени удаленного компьютера, с которым устанавливается связь) -> в поле *Username* указать имя пользователя OE -> в поле *Password* – пароль пользователя oe -> в поле Service Name записать строку идентификации удаленной базы данных в формате //<хост>:<порт>/<имя_службы>.
3. Переключиться на страницу DDL -> ознакомиться с оператором SQL создания связи базы данных -> OK (рис. 4.3).

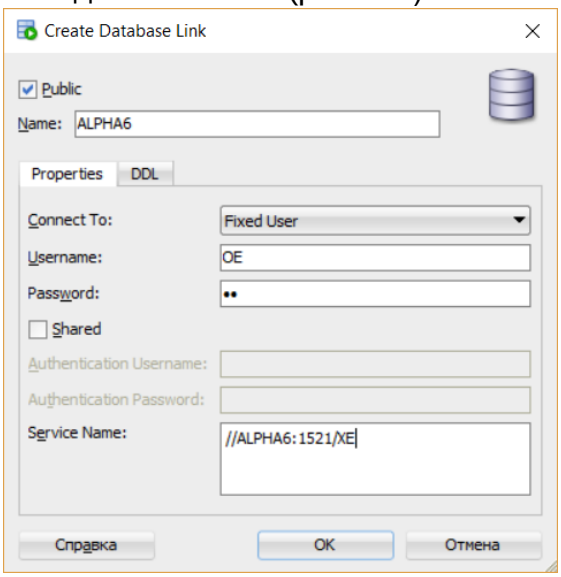


Рис. 4.3. Мастер создания связи базы данных

4. Проверить работу созданной связи. Для этого: в удаленной базе данных (с которой создана связь) в таблицу схемы OE OE.PROMOTIONS добавить новую запись -> зафиксировать транзакцию -> в локальной базе данных соединиться со схемой OE -> написать и выполнить SQL-запрос на выборку всех данных из удаленной таблицы OE.PROMOTIONS@<имя_связи> -> удостовериться, что в результате запроса присутствует новая запись с удаленной таблицы. Здесь использован синтаксис обращения к таблице удаленной базы данных по имени связи: <имя_таблицы>@<имя_связи>. Например, для связи с именем ALPHA6 имя запрашиваемой таблицы следует записать так: OE.PROMOTIONS@ALPHA6.

3.3.2. Представления в распределенных базах данных Oracle

Представление (view), или виртуальная таблица в базе данных Oracle Database — это специфический образ таблицы или набора таблиц, определенный оператором SELECT. Представление не существует физически как обычная таблица, являющаяся частью табличного пространства. Фактически представление создает виртуальную таблицу или подтаблицу только с теми строками и/или столбцами, которые нужно показать пользователю.

Представление Oracle — результат хранимого запроса, поэтому в словаре данных сохраняется только определение представления.

Представления Oracle используются в приложениях по нескольким причинам, включая перечисленные ниже:

- уменьшение сложности;
- повышение безопасности;
- повышение удобства с помощью объединения таблиц;
- переименование столбцов таблицы;
- настройка данных для пользователей;
- защита целостности данных.

При условии, что пользователь имеет соответствующие права доступа к лежащим в основе представления таблицам, можно запрашивать выборку данных из представления (SELECT) или даже модифицировать, удалять либо добавлять данные с использованием операторов UPDATE, DELETE и INSERT.

Представления создаются с помощью оператора SQL, описывающего композицию представления. При вызове представления выполняется запрос, по которому оно определено, и затем возвращается результат. Запрос, адресованный к представлению, выглядит в точности как обычный запрос, но база данных преобразует его в идентичный запрос к лежащим в его основе таблицам. Чтобы создать представление в своей схеме, необходимо иметь системную привилегию CREATE VIEW, а чтобы создать представление в любой схеме, а не только в собственной, понадобится системная привилегия CREATE ANY VIEW. Вдобавок нужно либо владеть лежащими в основе таблицами, либо иметь права на операции SELECT, INSERT, UPDATE и DELETE со всеми таблицами, на которых определено представление. Представление можно использовать для добавления к таблице мер безопасности уровня столбца или уровня значения. Безопасность уровня столбца обеспечивается созданием представлений, которые дают доступ лишь к избранным столбцам таблицы. Безопасность уровня значений включает применение конструкции WHERE в определении представления, которое отображает лишь избранные строки базовых таблиц. Чтобы использовать

представление, пользователю нужны привилегии для доступа к нему, а не к базовым таблицам, на которых оно определено.

Создание представлений средствами Oracle SQL Developer. Для создания представления необходимо выполнить два действия. Во-первых, написать и отладить SQL-запрос SELECT на выборку данных (как правило, с объединением нескольких связанных таблиц). Во-вторых, создать само представление как объект схемы пользователя – это удобно сделать с помощью мастера создания и редактирования представлений.

При написании и отладке SQL- запроса SELECT с объединением таблиц следует соблюдать несколько правил, облегчающих этот процесс.

1. Включение каждой очередной таблицы в запрос следует начинать с инструкции FROM, присваивая таблице псевдоним в обязательном порядке. Если используются таблицы из разных схем, имя таблицы обязательно должно иметь метку схемы (например, OE.CUSTOMERS cus, HR.EMPLOYEES emp). Если таблица находится в удаленной базе данных, в ее имя добавляется имя связи, как показано выше. Рекомендуется не включать в запрос следующую таблицу, пока не будут написаны все инструкции в секциях SELECT и WHERE запроса, относящиеся к очередной таблице.
2. В секции SELECT – перечислить отображаемые столбцы таблицы (с указанием её псевдонима), каждому столбцу рекомендуется давать русский псевдоним. Например, SELECT cu.cust_first_name "Имя заказч.".
3. В секции WHERE следует написать условие объединения очередной таблицы с одной из уже включенных в запрос по столбцам первичного и внешнего ключей. Отдельные условия в секции WHERE объединяются с помощью операции AND.
4. В зависимости от характера связи между таблицами, нужно использовать либо внутреннее, либо внешнее (левое или правое) объединение таблиц. Для реализации внешнего объединения таблиц рекомендуется использовать специальный оператор Oracle для внешних объединений – (+), который всегда записывается после имени столбца таблицы, которая в объединении является необязательной. Например, таблицы OE.CUSTOMERS и HR.EMPLOYEES объединяются с помощью левого внешнего объединения (главная таблица в этом объединении – это OE.CUSTOMERS, т.к. ее столбец внешнего ключа ACCOUNT_MGR_ID по связи с таблицей HR.EMPLOYEES допускает значения NULL). Условие левого внешнего объединения тогда запишется следующим образом:

   WHERE cus. ACCOUNT_MGR_ID = emp. EMPLOYEE.ID (+).
5. После включения в запрос очередной таблицы следует проверить правильность его выполнения и сохранить текст запроса (меню File -> Save или иконка Save на панели инструментов).

Пример: по рис. 4.2 для выборки из хранилища данных нужно объединить таблицы OE.ORDERS и OE.ORDER_ITEMS (вместе они составляют таблицу фактов), а также объединить с ними таблицы размерностей OE.CUSTOMERS и OE.PRODUCT_INFORMATION. SQL-запрос на объединение будет иметь следующий вид:

```
SELECT ord.*,
    oi.LINE_ITEM_ID, oi.PRODUCT_ID, oi.UNIT_PRICE, oi.QUANTITY,
```

```
        cu.CUST_FIRST_NAME, cu.CUST_LAST_NAME,
        pi.PRODUCT_NAME, pi.PRODUCT_STATUS
FROM OE.ORDERS ord,
    OE.ORDER_ITEMS oi,
    OE.CUSTOMERS cu,
    OE.PRODUCT_INFORMATION pi
WHERE ord.ORDER_ID = oi.ORDER_ID (+)
   AND cu.CUSTOMER_ID = ord.CUSTOMER_ID
   AND oi.PRODUCT_ID = pi.PRODUCT_ID;
```

Здесь цветами выделены этапы формирования запроса (черный -> зеленый -> синий -> красный).

Для создания представления на основе отлаженного запроса SELECT с помощью мастера SQL Developer следует:

1. Выбрать узел *Views* соединения OE -> контекстное меню -> New View… -> открывается форма мастера Create View.
2. В поле *Name* формы мастера написать уникальное содержательное имя представления (например, V_ORDER_ITEMS) -> на странице *SQL Query* в поле текстового редактора поместить текст запроса (вставить предварительно скопированный из редактора на странице *OE Worksheet*) -> кнопкой *Check Syntax* проверить синтаксис запроса -> кнопкой *Test…* проверить правильность выполнения запроса.
3. На странице *Properties* установить флажок *Force On Create* -> на странице *DDL* можно увидеть текст оператора SQL создания представления (обратите внимание, что это оператор именно только создания, начинающийся с ключевых слов CREATE FORCE VIEW) -> OK (рис. 4.4).

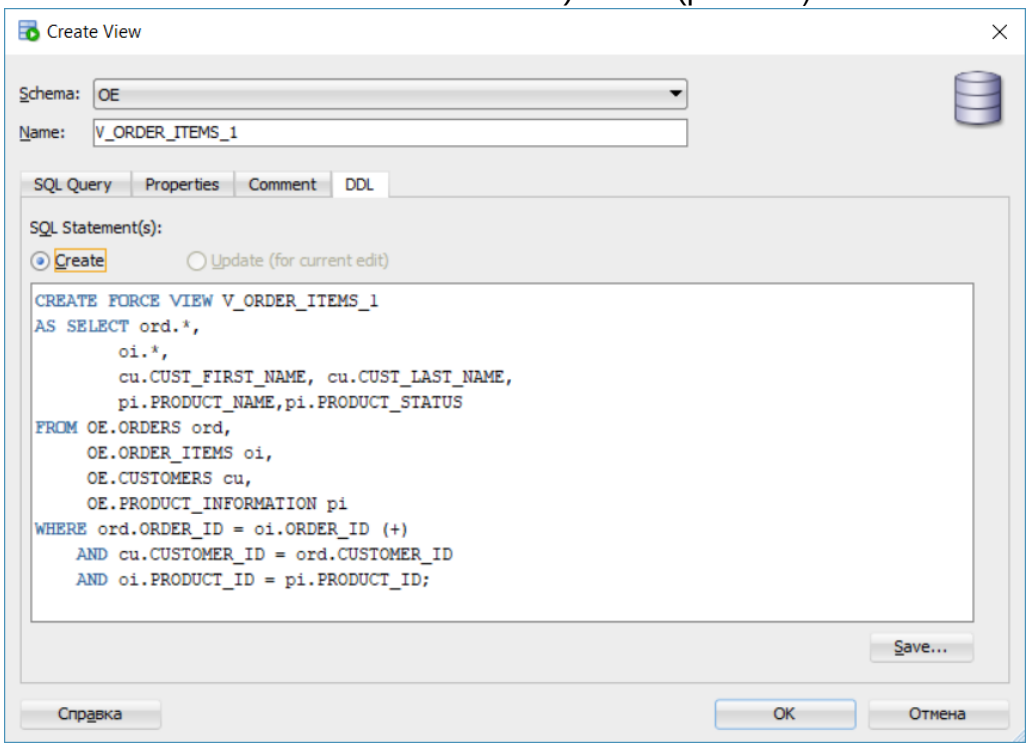


Рис. 4.4. Мастер создания представления

После создания представление должно появиться в списке *Views* соединения OE. Щелкнув на него, можно просмотреть все его свойства. **Для редактирования представления** нужно выбрать его в списке *Views* -> контекстное меню -> Edit… -> открывается форма мастера, аналогичная рис. 4.4 -> на странице *SQL Query* внести

изменения в текст запроса -> на странице *Properties* установить флажок *Force On Create* -> на странице *DDL* выбрать режим *Update (for current edit)* -> можно просмотреть текст оператора SQL измененного представления (обратите внимание, что это оператор создания или замены существующего представления, начинающийся с ключевых слов CREATE OR REPLACE FORCE VIEW) -> OK.
Добавление к оператору CREATE VIEW конструкции WITH READ ONLY гарантирует, что пользователи смогут только осуществлять выборку данных из представления. Это означает, что пользователи не смогут модифицировать представление и тем самым неявно обновлять, вставлять или удалять строки базовых таблиц. В противном случае по умолчанию Oracle позволяет обновлять представление.

3.3.3. Создание материализованных представлений в распределенных базах данных Oracle

Материализованные представления — это объекты схем, которые могут применяться для обобщения, предварительного обсчета, тиражирования и распределения данных. В отличие от обычного представления, которое содержит лишь SQL запрос, материализованное представление сохраняет строки данных, полученные в результате выполнении SQL-запроса, относящегося к одной или нескольким исходным таблицам.
Имеется возможность установить автоматическую синхронизацию представления с исходными таблицами через интервалы времени, определенные пользователем. Материализованные представления могут сохраняться в той же базе данных, что и исходные таблицы, либо в другой, удаленной базе данных.
Материализованные представления могут использоваться в хранилищах данных. Часто они применяются для предварительного вычисления и хранения агрегированных данных, таких как суммы и средние значения. Пусть, например, материализованное представление содержит данные о месячных объемах продаж. Тогда при вводе в исходную таблицу новых данных о продажах в течение месяца материализованное представление автоматически обновит итоговые суммы, которые будут отражать последнюю информацию, содержащуюся в исходных таблицах. Сохранение предварительно обсчитанной и обобщенной информации позволяет значительно повысить скорость выполнения запросов в больших хранилищах данных.
В распределенном окружении можно использовать материализованные представления для реплицирования данных, хранящихся на удаленных серверах базы данных, и для синхронизации обновлений на этих серверах. В среде мобильных вычислений материализованные представления позволяют загрузить на мобильный клиентский компьютер подмножество данных с центрального сервера и периодически выполнить синхронизацию обновлений на сервере и клиенте.
Для создания материализованного представления необходимы следующие пять элементов:

1. Основной узел. Здесь находится одна или несколько таблиц, чьи данные служат источником для моментального снимка.
2. Исходная таблица (таблицы). Эта таблица содержит данные, представляемые в виде моментального снимка.

3. Журнал материализованных представлений исходной таблицы на основном узле (создается отдельно для каждой исходной таблицы). Этот журнал предназначен для локального сохранения транзакций, взаимодействующих с исходными таблицами.
4. Очередь рабочих процессов Oracle. Процессы, поставленные в эту очередь, передают содержимое журнала материализованных представлений на удаленный узел.
5. Удаленный узел. Здесь обрабатываются транзакции по мере их поступления и выполняется обновление, удаление или добавление новых строк в материализованное представление.

Для ввода в действие материализованного представления необходимо выполнить следующие три шага:

1. Выдать необходимые привилегии.
2. Создать журнал материализованного представления.
3. Создать материализованное представление.

Системные привилегии пользователей. Чтобы иметь возможность создавать журнал материализованного представления, владелец одной или нескольких таблиц на основном узле, которые служат источником данных для материализованного представления, должен иметь специальную системную привилегию CREATE MATERIALIZED VIEW. Чтобы иметь возможность создавать материализованное представление на удаленном узле, его владелец должен иметь ту же системную привилегию CREATE MATERIALIZED VIEW. Выдача этих системных привилегий может производиться, например, от имени пользователей *sys* или *system* с помощью утилиты SQL*Plus (SQL Command Line) командой

SQL > GRANT CREATE MATERIALIZED VIEW TO OE;

Создание журнала материализованного представления. Журнал материализованного представления на основном узле удобно создавать с помощью мастера Oracle SQL Developer. Для этого:

1. В навигаторе *Connections* раскрыть соединение пользователя (например, OE) -> выбрать узел *Materialized View Logs* -> контекстное меню -> New Materialized View Log… -> открывается форма мастера Create Materialized View Log.
2. В поле *Master Table* выбрать из выпадающего списка требуемую таблицу, для которой создается журнал (например, ORDERS) -> в полях *Rowid Logged:, Primary Key Logged:, Sequence Logged:* выбрать из выпадающего списка значение Yes -> в поле *New Values:* выбрать из выпадающего списка значение Including.
3. В поле *Selected Columns* перенести из поля *Availaible Columns* имена столбцов, которые будут участвовать в материализованном представлении (обычно все столбцы) -> на странице *DDL* можно посмотреть текст SQL-команды создания журнала -> OK (рис. 4.5).

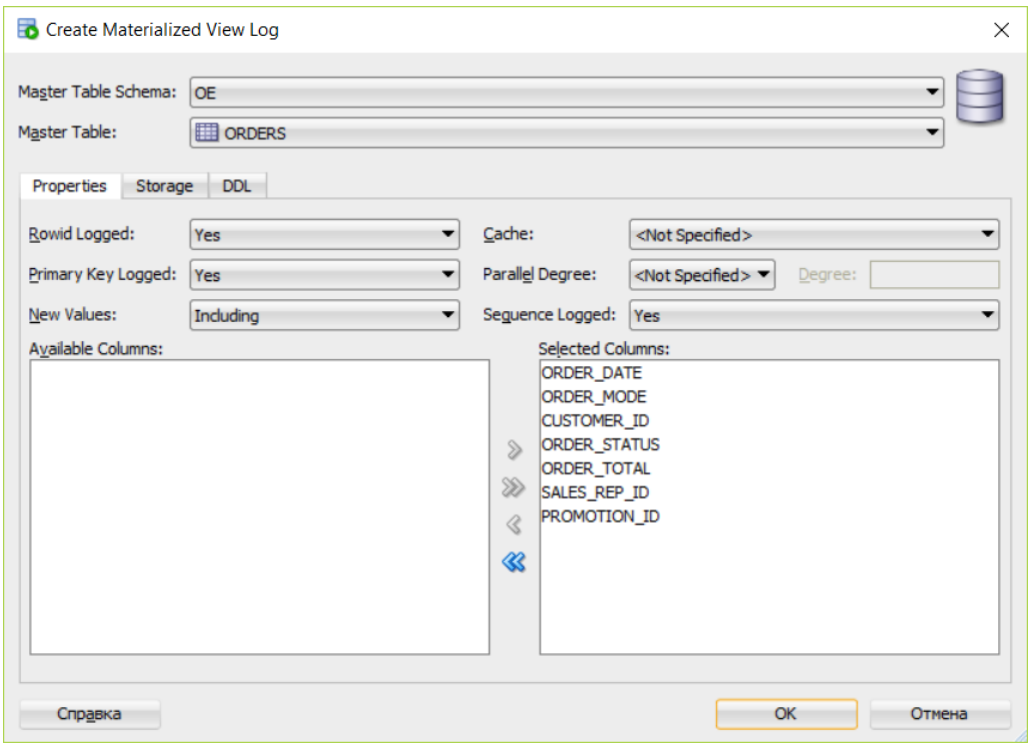


Рис. 4.5. Мастер создания журнала материализованного представления

Поскольку свободно распространяемая редакция базы данных Oracle Database 11g Express Edition не поддерживает материализованные представления, будет выдано сообщение о невозможности выполнения операции.

Создание материализованного представления. Материализованное представления на удаленном узле удобно создавать с помощью мастера Oracle SQL Developer. Для этого:

1. В навигаторе *Connections* раскрыть соединение пользователя (например, OE) -> выбрать узел *Materialized Views* -> контекстное меню -> New Materialized View… -> открывается форма мастера Create Materialized View.
4. В поле *Name* задать уникальное содержательное имя материализованного представления, например, MV_ORDER_ITEMS -> на странице *SQL Query* в поле текстового редактора поместить текст запроса (вставить предварительно скопированный из редактора на странице *OE Worksheet*) -> кнопкой *Check Syntax* проверить синтаксис запроса -> кнопкой *Test…* проверить правильность выполнения запроса.
2. На странице *Refresh Options*:
   - Для параметра *Refresh Mode:* установите значение ON COMMIT (в этом режиме при всякой фиксации изменений данных в главных таблицах материализованное представление обновляется автоматически, отражая эти изменения.
   - Для параметра *Refresh Method:* установите значение Fast либо Force. Для реализации механизма быстрого обновления Fast Oracle использует журнал материализованного представления для регистрации всех изменений в главных таблицах. Затем журнал материализованного представления применяется для обновления этого материализованного представления. В случае выбора этой опции Force Oracle попытается применить механизм быстрого обновления (fast refresh). Если по некоторым причинам он не может быть использован, применяется метод полного обновления.

- Для параметра *Key Type:* установите значение Row ID.
- Для параметра *Using Constraints:* установите значение Enforced.

3. На странице *Properties*:
   - Для параметра *Query Rewrite:* установите значение Enabled. Этот параметр определяет **переписывание запросов**. В крупных базах данных Oracle с интенсивными действиями, затратными по времени и вычислительной мощности процессоров, такими как соединение таблиц и использование агрегатных функций вроде SUM, материализованные представления обеспечивают более быстрое выполнение запросов за счет перерасчета и хранения результатов дорогостоящих соединений и агрегатных операций. Достоинство материализованных представлений Oracle заключается в том, что при их создании можно указать, что база данных должна автоматически обновлять материализованные представления, когда происходят изменения в положенных в их основу таблицах. Материализованные представления полностью прозрачны для пользователей. Если пользователи пишут запросы с обращением к лежащим в основе таблицам, то Oracle автоматически переписывает их для использования материализованных представлений, и такая техника оптимизации запросов называется переписыванием запроса (query rewrite). Стоимостной оптимизатор Oracle автоматически распознает необходимость в переписывании запроса для использования материализованного представления вместо исходных таблиц, если оценочная стоимость такого запроса оказывается ниже. Под стоимостью запроса здесь подразумевается объем ввода-вывода, а также затраты времени процессора и памяти, связанные с обработкой SQL-запроса. Сложные соединения таблиц обходятся в этом смысле дорого, а применение материализованных представлений позволяет использовать уже сохраненную информацию в предварительно вычисленном виде, и запросы требуют гораздо меньше ресурсов и потому выполняются намного быстрее. Прием автоматической оптимизации с переписыванием запроса лежит в основе применения материализованных представлений. Параметр инициализации QUERY_REWRITE_ENABLED позволяет включать и отключать это средство на глобальном уровне.
   - Для параметра *Build Type:* установите значение Immediate.
4. На странице *Storage* установите флажок *Define Storage* -> для параметра *Tablespace:* установите значение USERS -> для параметра *Initial Extent:* установите значение 8 и K из выпадающего списка -> для параметра *Next Extent:* установите так же значение 8K -> установите флажок *Unlimited.*
5. На странице *DDL* можно посмотреть текст SQL-команды создания материализованного представление -> OK (рис. 4.6).

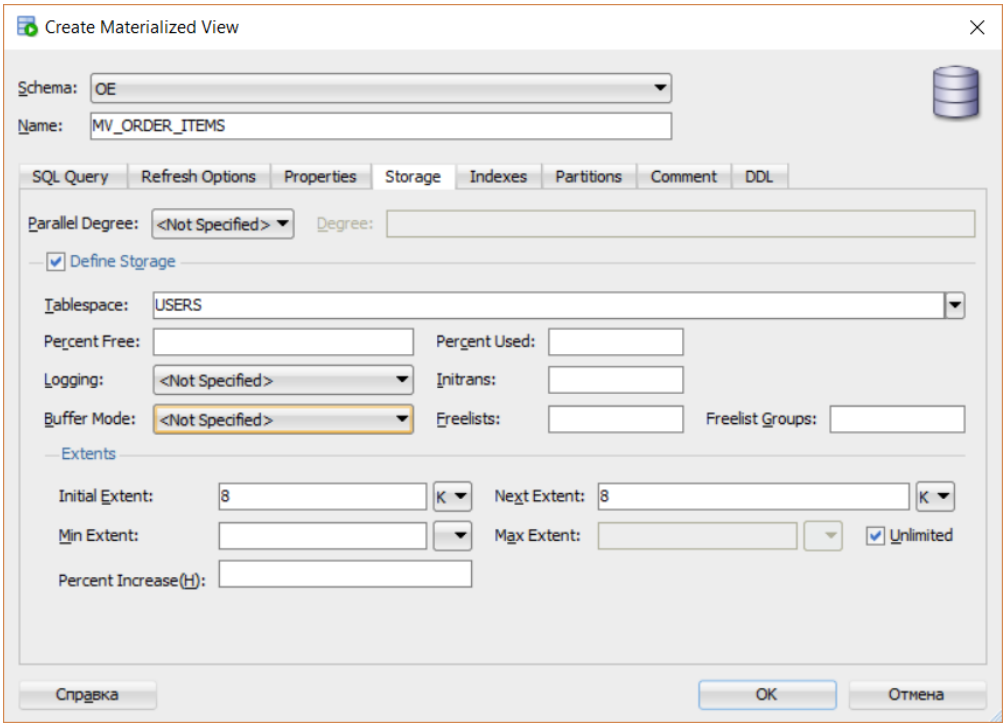


Рис. 4.6. Мастер создания материализованного представления

Поскольку свободно распространяемая редакция базы данных Oracle Database 11g Express Edition не поддерживает материализованные представления, будет выдано сообщение о невозможности выполнения операции.

3. Меры безопасности

Во время выполнения лабораторной работы необходимо:

- соблюдать правила включения и выключения вычислительной техники;
- не подключать кабели, разъемы и другую аппаратуру к компьютеру, не относящиеся к лабораторной установке;
- при включенном напряжении сети не отключать, не подключать и не трогать кабели, соединяющие различные устройства компьютера;
- в случае обнаруженной неисправности в работе оборудования или нарушения правил техники безопасности сообщить руководителю лабораторной работы;
- не пытаться самостоятельно устранить неисправности в работе аппаратуры;
- по окончании работы привести в порядок рабочее место.

ВНИМАНИЕ! При работе за компьютером необходимо помнить: к каждому рабочему месту подведено опасное для жизни напряжение. Поэтому во время работы надо быть предельно внимательным и соблюдать все требования техники безопасности!

4. Задание

1. С помощью утилиты SQL*Plus подключиться к базе данных под именем администратора SYS и предоставить пользователю OE системную привилегию CREATE PUBLIC DATABASE LINK.

2. Выбрать доступный для удаленного подключения компьютер, найти и открыть на нем файл tnsnames.ora, определить параметры удаленного подключения к базе данных Oracle 11g XE.
3. С помощью утилиты SQL*Plus на рабочем компьютере подключиться к удаленной базе данных под именем пользователя OE и удостовериться в успешности подключения.
4. С помощью мастера SQL Developer создать общедоступную связь с удаленной базой данных. Проверить работу созданной связи.
5. Средствами SQL Developer создать представление для выборки данных из связанных таблиц схемы OE ORDERS, ORDER_ITEMS, CUSTOMERS в соответствии со схемой рис. 4.2. Подсчитать количество результирующих строк, возвращаемых представлением.
6. Отредактировать созданное представление путем подключения связанной таблицы PRODUCT_INFORMATION. Подсчитать количество результирующих строк, возвращаемых представлением.
7. Отредактировать созданное представление путем подключения двух связанных таблиц EMPLOYEES с разными псевдонимами: одну объединить с таблицей CUSTOMERS, другую – с таблицей ORDERS (внешнее объединение, таблица EMPLOYEES в нем необязательная). Подсчитать количество результирующих строк, возвращаемых представлением.
8. Отредактировать созданное представление путем изменения подключения связанной таблицы PRODUCT_INFORMATION – вместо локальной таблицы использовать таблицу из удаленной базы данных. Подсчитать количество результирующих строк, возвращаемых представлением. Проверить работу представления с помощью изменения наименования товара (PRODUCT_NAME) в удаленной таблице – эти изменения должны отразиться в возвращаемых представлением данных.
9. Выполнить процедуру создания журнала материализованного представления.
10. Выполнить процедуру создания материализованного представления.
11. Удалить созданное представление.

5. Требования к содержанию и оформлению отчета

Отчет должен быть выполнен в текстовом редакторе MS Word. Отчет должен содержать:

- Краткие теоретические сведения.
- Тексты всех SQL-команд с комментариями относительно каждой использованной в запросе инструкции.
- Скриншоты выполнения всех SQL-команд.
- Результирующие таблицы с данными для всех выполненных запросов и команд.
- Выводы по проделанной работе.

6. Контрольные вопросы

1. Что такое распределенная база данных?
2. Каковы принципы и правила создания распределенных баз данных?
3. Что такое хранилище данных?
4. Каковы основные принципы создания хранилищ данных?
5. Какие способы разделения данных используются в хранилищах данных?
6. Что такое связь базы данных?
7. Как создаются связи баз данных?
8. Как создаются представления в распределенных базах данных?
9. Что такое материализованное представление?
10. С какими целями используются материализованные представления?
11. Как создаются материализованные представления в распределенных базах данных?